

# Parcial 1 — Parte 1 — Minería de Datos

Profesor: Pierre Rosado | Universidad del Norte

Dataset: Online Retail II (UCI ML Repository)

Autor: desarrollo\_evol\_4@jamar.com

Este documento resume las respuestas y evidencias generadas en el notebook

**Parcial1\_Parte1\_MineriaDeDatos.ipynb** (adjunto/entregado por separado en Google Colab), que contiene la carga de datos, el EDA, la limpieza y la construcción de canastas (baskets) sobre el dataset Online Retail II.

## 1. Descarga de datos

Se descargó el archivo **online+retail+ii.zip** desde <https://archive.ics.uci.edu/dataset/502/online+retail+ii>, se descomprimió y se inspeccionó el archivo Excel **online\_retail\_II.xlsx**, que contiene dos hojas: *Year 2009-2010* y *Year 2010-2011*, correspondientes a transacciones de una tienda online del Reino Unido entre el 01/12/2009 y el 09/12/2011.

## 2. Cargar los datos y EDA

Evidencia — código: carga del Excel

```
In [2]:  
  
xls = pd.ExcelFile(DATA_PATH)  
print("Hojas del archivo Excel:", xls.sheet_names)  
  
df = pd.concat(  
    [pd.read_excel(xls, sheet_name=s) for s in xls.sheet_names],  
    ignore_index=True  
)  
df.head()
```

### Pregunta 2.1 — Columnas de canastas e ítems

- a) ¿Cuál es la columna que identifica a las canastas?
- b) ¿Cuál es la columna que identifica a los ítems?

```
In [3]:  
  
print("a) Columna que identifica las canastas: 'Invoice'")  
print("b) Columna que identifica los ítems: 'StockCode'")
```

**Respuesta:** a) **Invoice** identifica la canasta (cada factura agrupa los ítems comprados juntos). b) **StockCode** identifica el ítem (código único de producto).

## Pregunta 2.2 — Filas originales

¿Cuántas filas tiene el dataset originalmente?

```
In [4]:  
  
n_filas_originales = len(df)  
print("Filas originales:", n_filas_originales)
```

**Respuesta:** El dataset original (unión de ambas hojas) tiene **1,067,371** filas.

## Pregunta 2.3 — EDA

- a) ¿Cuántas filas tienen valores nulos en Invoice o StockCode?
- b) ¿Cuántas filas tienen un Invoice que empieza con "C"?
- c) Valores de StockCode que no siguen el patrón de 5 dígitos con sufijo de letra opcional (se esperan 68 valores únicos).

Evidencia — código apartado a):

```
In [5]:  
  
# a) Nulos en Invoice o StockCode  
nulos_invoice_o_stock = df["Invoice"].isna() | df["StockCode"].isna()  
print("a) Filas con nulos en Invoice o StockCode:", nulos_invoice_o_stock.sum())
```

Evidencia — código apartado b):

```
In [6]:  
  
# b) Invoices que empiezan con 'C' (cancelaciones)  
invoice_str = df["Invoice"].astype(str)  
cancelaciones = invoice_str.str.startswith("C")  
print("b) Filas con Invoice que empieza con 'C':", cancelaciones.sum())
```

Evidencia — código apartado c):

```
In [7]:  
  
# c) StockCodes que no siguen el patrón 5 dígitos + sufijo de letra opcional  
patron = re.compile(r"^\d{5}[A-Za-z]?")  
stockcode_str = df["StockCode"].astype(str)  
no_cumple_patron = ~stockcode_str.str.match(patron)  
  
stockcodes_atipicos = sorted(stockcode_str[no_cumple_patron].unique().tolist())  
print("c) Cantidad de StockCodes que no siguen el patrón:", len(stockcodes_atipicos))  
stockcodes_atipicos
```

**Respuesta:**

- a) **0** filas tienen nulos en Invoice o StockCode.
- b) **19,494** filas tienen un Invoice que empieza con "C" (cancelaciones).
- c) Se identificaron **68** valores únicos de StockCode que no siguen el patrón (coincide con el valor esperado de 68).

**Evidencia — listado de los 68 StockCodes atípicos:**

15056BL, 15056bL, 47503J , 72024HC, 79323GR, 79323LP, ADJUST, ADJUST2, AMAZONFEE, B, BANK CHARGES, C2, C3,

### 3. Limpieza de datos

Reglas de limpieza aplicadas: eliminar cancelaciones (Invoice que empieza con "C"), conservar solo filas con Quantity > 0, eliminar los stock codes no-producto (POST, D, DOT, M, S, AMAZONFEE, BANK CHARGES, PADS, CRUK, C2, m), eliminar stock codes de longitud 1, y eliminar filas con Customer ID nulo.

Evidencia — código de limpieza:

```
In [8]:

df_clean = df.copy()
df_clean["Invoice"] = df_clean["Invoice"].astype(str)
df_clean["StockCode"] = df_clean["StockCode"].astype(str)

# Eliminar cancelaciones (Invoice empieza con 'C')
df_clean = df_clean[~df_clean["Invoice"].str.startswith("C")]

# Conservar solo Quantity > 0
df_clean = df_clean[df_clean["Quantity"] > 0]

# Eliminar stock codes no-producto
non_product_codes = {
    "POST", "D", "DOT", "M", "S", "AMAZONFEE",
    "BANK CHARGES", "PADS", "CRUK", "C2", "m"
}
df_clean = df_clean[~df_clean["StockCode"].isin(non_product_codes)]

# Eliminar stock codes de longitud 1
df_clean = df_clean[df_clean["StockCode"].str.len() != 1]

# Eliminar filas con Customer ID nulo
df_clean = df_clean[df_clean["Customer ID"].notna()]

print("Filas tras limpieza:      ", len(df_clean))
print("StockCodes únicos:      ", df_clean["StockCode"].nunique())
print("Invoices únicos:         ", df_clean["Invoice"].nunique())
```

Evidencia — código de verificación:

```
In [9]:

# Verificación contra los valores esperados por el enunciado
assert len(df_clean) == 802_742
assert df_clean["StockCode"].nunique() == 4_624
assert df_clean["Invoice"].nunique() == 36_644
print("Verificación OK: 802,742 filas | 4,624 StockCodes únicos | 36,644 Invoices únicos")
```

| Métrica             | Esperado | Obtenido | OK |
|---------------------|----------|----------|----|
| Filas tras limpieza | 802,742  | 802,742  | ✓  |
| StockCodes únicos   | 4,624    | 4,624    | ✓  |
| Invoices únicos     | 36,644   | 36,644   | ✓  |

**Verificación:** los tres valores coinciden exactamente con los esperados en el enunciado.

## 4. Construir los baskets

### 4.1 — *Importancia de frozenset*

**Respuesta:** `frozenset` es la versión inmutable de `set` en Python. Es importante para representar los ítems de una canasta porque: (1) al ser inmutable y *hashable*, puede usarse como clave de diccionario o como elemento de otro set — indispensable para algoritmos de reglas de asociación (Apriori/FP-Growth) que cuentan itemsets como claves; (2) elimina duplicados y no depende del orden de los ítems, que es la semántica correcta de una canasta de compra; (3) permite operaciones eficientes de conjuntos (unión, intersección, subconjunto); y (4) evita modificaciones accidentales de la canasta durante el procesamiento, garantizando resultados reproducibles.

## 4.2 — Construcción de canastas

Evidencia — código:

```
In [10]:  
  
baskets_series = df_clean.groupby("Invoice")["StockCode"].apply(lambda s: frozenset(s))  
  
n_canastas = len(baskets_series)  
print("Total de canastas:", n_canastas)  
assert n_canastas == 36_644
```

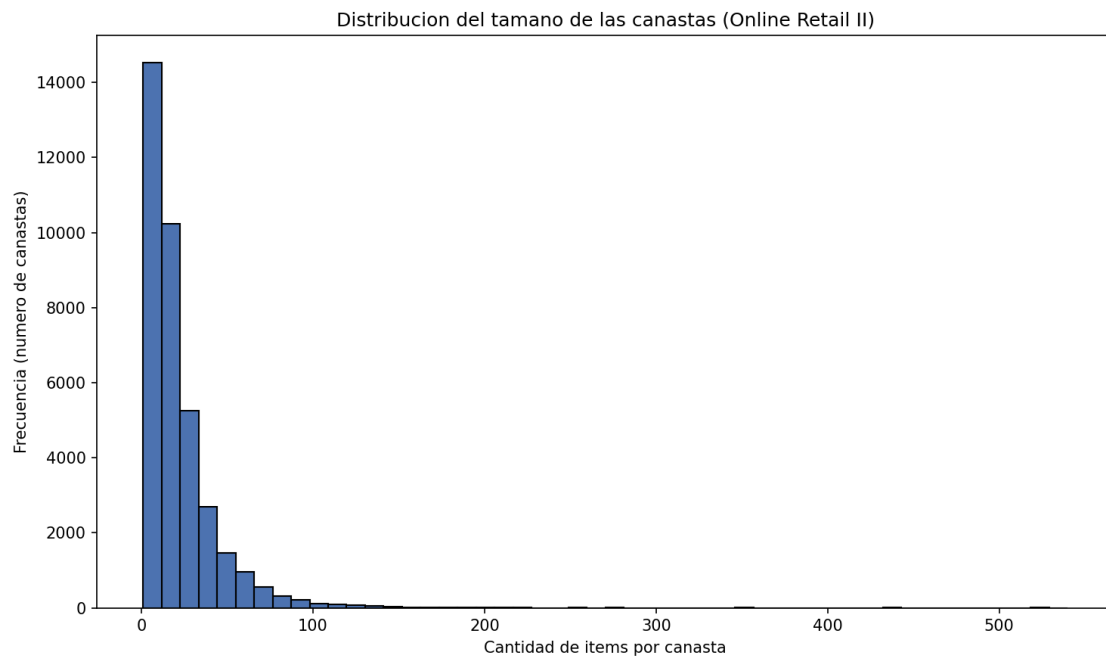
**Respuesta:** se agruparon los StockCode por Invoice como frozenset, obteniendo **36,644** canastas, coincidiendo con el valor esperado de 36,644.

## 4.3 — Distribución de tamaños de canastas

Evidencia — código:

```
In [11]:  
  
basket_sizes = baskets_series.apply(len)  
  
plt.figure(figsize=(10, 6))  
plt.hist(basket_sizes, bins=50, color="#4C72B0", edgecolor="black")  
plt.title("Distribución del tamaño de las canastas (Online Retail II)")  
plt.xlabel("Cantidad de ítems por canasta")  
plt.ylabel("Frecuencia (número de canastas)")  
plt.tight_layout()  
plt.savefig("basket_size_histogram.png", dpi=150)  
plt.show()  
  
print("Tamaño mínimo: ", basket_sizes.min())  
print("Tamaño máximo: ", basket_sizes.max())  
print("Tamaño promedio:", round(basket_sizes.mean(), 2))  
print("Tamaño mediana: ", basket_sizes.median())
```

Tamaño mínimo: 1 ítems | Tamaño máximo: 540 ítems | Promedio: 20.91 ítems | Mediana: 15 ítems



#### 4.4 — Exportación a CSV

Evidencia — código:

```
In [12]:  
  
# Guardar las canastas en un archivo CSV  
baskets_df = pd.DataFrame({  
    "Invoice": baskets_series.index,  
    "Items": baskets_series.apply(lambda fs: ",".join(sorted(fs)))  
})  
baskets_df.to_csv("baskets.csv", index=False)  
baskets_df.head()
```

Las canastas se guardaron en **baskets.csv** (una fila por Invoice, con la columna Items conteniendo los StockCode separados por coma).

## Parcial 1 — Parte 2 — Minería de Datos

Continuando con las canastas construidas en la Parte 1, se implementa el algoritmo **A-Priori** y la **generación de reglas de asociación**.

### Ejemplo sintético (verificación manual)

Antes de correr el algoritmo sobre el dataset completo, se probó la implementación con un ejemplo pequeño y sintético (6 canastas de "pan", "leche", "pañales", "cerveza", "huevos") que se puede verificar a mano. Con soporte mínimo = 3 se espera **L1** = {pan, leche, pañales, cerveza} (huevos queda fuera, soporte 1) y **L2** = {(pan,leche), (pan,pañales), (leche,pañales), (pañales,cerveza)}.

Evidencia — código y salida de la ejecución:

```
In [14]:  
  
# Ejemplo sintético de 6 canastas, calculable manualmente  
canastas_ejemplo = [  
    frozenset(["pan", "leche", "huevos"]),  
    frozenset(["pan", "leche"]),  
    frozenset(["pan", "pañales", "cerveza"]),  
    frozenset(["leche", "pañales", "cerveza"]),  
    frozenset(["pan", "leche", "pañales", "cerveza"]),  
    frozenset(["pan", "leche", "pañales"]),  
]  
  
# Con min_support=3 (soporte >= 3 de 6 canastas):  
# Conteos individuales -> pan:5, leche:5, pañales:4, cerveza:3, huevos:1  
# L1 esperado (soporte>=3): {pan, leche, pañales, cerveza}  
# Pares con soporte>=3 (contados a mano): (pan,leche)=4, (pan,pañales)=3,  
#   (leche,pañales)=3, (pañales,cerveza)=3  
# L2 esperado: esos 4 pares  
freq_ejemplo, candidatos_ejemplo = apriori(canastas_ejemplo, min_support=3, max_k=4)  
  
for k in sorted(freq_ejemplo):  
    print(f"L{k}:", {tuple(sorted(iset)): cnt for iset, cnt in freq_ejemplo[k].items()})
```

**Resultado:** la salida de la función coincide exactamente con el conteo manual, confirmando que la implementación es correcta antes de aplicarla al dataset completo.

# 1. Algoritmo A-Priori

## *Implementación*

Se implementó la función **apriori(baskets, min\_support, max\_k=4)** siguiendo los pasos vistos en clase: Pasada 1 cuenta ítems individuales y filtra por soporte mínimo para obtener L1; en cada Pasada k se generan candidatos Ck a partir de Lk-1 (join + poda por subconjuntos, función auxiliar **generate\_candidates**), se cuenta su soporte escaneando las canastas y se filtra para obtener Lk. El proceso se repite hasta que no se generen más itemsets frecuentes o se alcance max\_k.

Evidencia — código:



```

In [13]:

import math
from collections import defaultdict
from itertools import combinations

def generate_candidates(prev_itemsets, k):
    """Genera candidatos Ck a partir de Lk-1 (join + poda por subconjuntos)."""
    prev_set = set(prev_itemsets)
    sorted_prev = [tuple(sorted(iset)) for iset in prev_itemsets]
    candidates = set()
    n = len(sorted_prev)
    for i in range(n):
        for j in range(i + 1, n):
            a, b = sorted_prev[i], sorted_prev[j]
            if a[:k - 2] == b[:k - 2]:
                candidate = frozenset(a) | frozenset(b)
                if len(candidate) == k:
                    if all(frozenset(c) in prev_set for c in combinations(candidate, k - 1)):
                        candidates.add(candidate)
    return candidates

def apriori(baskets, min_support, max_k=4):
    """
    Algoritmo A-Priori.

    baskets: Lista de frozensets
    min_support: soporte mínimo absoluto (número de baskets)
    max_k: tamaño máximo de itemsets a buscar

    Retorna: dict {k: {frozenset: count}}
    """
    freq = {}
    candidates_count = {}

    # Pasada 1: contar ítems individuales
    item_counts = defaultdict(int)
    for b in baskets:
        for item in b:
            item_counts[item] += 1
    C1 = {frozenset([item]): cnt for item, cnt in item_counts.items()}
    candidates_count[1] = len(C1)

    L1 = {iset: cnt for iset, cnt in C1.items() if cnt >= min_support}
    freq[1] = L1

    k = 2
    Lk_minus_1 = L1
    while Lk_minus_1 and k <= max_k:
        prev_itemsets = list(Lk_minus_1.keys())
        items_in_prev = set()
        for iset in prev_itemsets:
            items_in_prev |= iset

        Ck = generate_candidates(prev_itemsets, k)
        candidates_count[k] = len(Ck)
        if not Ck:
            break

        counts = defaultdict(int)
        for b in baskets:
            reduced = b & items_in_prev
            if len(reduced) < k:
                continue
            for combo in combinations(sorted(reduced), k):
                fs = frozenset(combo)
                if fs in Ck:
                    counts[fs] += 1

        Lk = {iset: cnt for iset, cnt in counts.items() if cnt >= min_support}
        freq[k] = Lk

        Lk_minus_1 = Lk
        k += 1

    return freq, candidates_count

```

## Ejecución

Evidencia — código: preparación de baskets y descripciones de producto:

```
In [15]:

# Reconstruir la lista de baskets (frozensets) a partir de La Parte 1
baskets = list(baskets_series.values)
n_baskets = len(baskets)

# Mapeo StockCode -> Descripción (La más frecuente para ese StockCode)
desc_map = (
    df_clean.groupby("StockCode")["Description"]
    .agg(lambda s: s.mode().iloc[0] if not s.mode().empty else s.iloc[0])
    .to_dict()
)

def describe(item):
    return desc_map.get(item, item)
```

Evidencia — código: cálculo del soporte mínimo y ejecución del algoritmo:

```
In [16]:

min_support = math.floor(0.02 * n_baskets)
print("Soporte mínimo absoluto (2% de", n_baskets, "canastas):", min_support)

freq, candidates_count = apriori(baskets, min_support, max_k=4)
```

Evidencia — código y salida: reporte de candidatos y frecuentes por pasada:

```
In [17]:

print("Reporte por pasada:")
total_frequent = 0
for k in sorted(set(list(candidates_count.keys()) + list(freq.keys()))):
    n_candidates = candidates_count.get(k, 0)
    n_frequent = len(freq.get(k, {}))
    total_frequent += n_frequent
    print(f"  k={k}: candidatos C{k}={n_candidates}, frecuentes L{k}={n_frequent}")

print("\nTotal de itemsets frecuentes:", total_frequent)

assert min_support == 732
assert len(freq[1]) == 196
assert len(freq[2]) == 31
assert total_frequent == 227
print("\nVerificación OK: soporte mínimo 732 | L1=196 | L2=31 | total=227")
```

| k | Candidatos C <sub>k</sub> | Frecuentes L <sub>k</sub> |
|---|---------------------------|---------------------------|
| 1 | 4,624                     | 196                       |
| 2 | 19,110                    | 31                        |
| 3 | 18                        | 0                         |

**Respuesta:** con soporte mínimo de **732** (2.0% de 36,644 canastas), se obtienen **196** ítems frecuentes en L1, **31** pares frecuentes en L2 y 0 tríos frecuentes en L3 (el algoritmo se detiene antes de max\_k=4 al no generarse más itemsets frecuentes), para un total de **227** itemsets frecuentes — coincidiendo

exactamente con la verificación del enunciado.

## Los 15 pares frecuentes con mayor soporte:

Evidencia — código:

```
In [18]:  
  
# Top 15 pares frecuentes con mayor soporte, con descripción de productos  
pairs = freq[2]  
top15 = sorted(pairs.items(), key=lambda x: x[1], reverse=True)[:15]  
  
top15_rows = []  
for iset, cnt in top15:  
    items = sorted(iset)  
    top15_rows.append({  
        "Item A": items[0], "Descripción A": describe(items[0]),  
        "Item B": items[1], "Descripción B": describe(items[1]),  
        "Conteo": cnt, "Soporte": round(cnt / n_baskets, 4),  
    })  
  
top15_df = pd.DataFrame(top15_rows)  
top15_df
```

| Producto A                        | Producto B                         | Conteo | Soporte |
|-----------------------------------|------------------------------------|--------|---------|
| RED HANGING HEART T-LIGHT HOLDER  | WHITE HANGING HEART T-LIGHT HOLDER | 1,155  | 0.0315  |
| JUMBO BAG PINK POLKADOT           | JUMBO BAG RED RETROSPOT            | 1,064  | 0.0290  |
| LUNCH BAG RED RETROSPOT           | LUNCH BAG PINK POLKADOT            | 1,022  | 0.0279  |
| LUNCH BAG RED RETROSPOT           | LUNCH BAG BLACK SKULL.             | 1,009  | 0.0275  |
| WOODEN PICTURE FRAME WHITE FINISH | WOODEN FRAME ANTIQUE WHITE         | 993    | 0.0271  |
| LUNCH BAG RED RETROSPOT           | LUNCH BAG SUKI DESIGN              | 976    | 0.0266  |
| JUMBO BAG RED RETROSPOT           | JUMBO BAG STRAWBERRY               | 952    | 0.0260  |
| PACK OF 72 RETROSPOT CAKE CASES   | 60 TEATIME FAIRY CAKE CASES        | 913    | 0.0249  |
| JUMBO STORAGE BAG SUKI            | JUMBO BAG RED RETROSPOT            | 913    | 0.0249  |
| LUNCH BAG SPACEBOY DESIGN         | LUNCH BAG SUKI DESIGN              | 909    | 0.0248  |
| LUNCH BAG RED RETROSPOT           | LUNCH BAG CARS BLUE                | 903    | 0.0246  |
| LUNCH BAG RED RETROSPOT           | LUNCH BAG SPACEBOY DESIGN          | 886    | 0.0242  |
| HEART OF WICKER SMALL             | HEART OF WICKER LARGE              | 869    | 0.0237  |
| SWEETHEART CERAMIC TRINKET BOX    | STRAWBERRY CERAMIC TRINKET BOX     | 857    | 0.0234  |

| Producto A              | Producto B         | Conteo | Soporte |
|-------------------------|--------------------|--------|---------|
| LUNCH BAG RED RETROSPOT | LUNCH BAG WOODLAND | 856    | 0.0234  |

## 2. Generación de reglas de asociación

### *Implementación*

Se implementó **generate\_rules(freq, support, n\_baskets, min\_confidence, min\_interest)**: para cada itemset frecuente  $I$  de tamaño  $\geq 2$  se generan todas las reglas  $A \rightarrow B$  con  $A \cup B = I$  y  $A \cap B = \emptyset$ , calculando soporte, confianza, lift e interés, y reteniendo solo las reglas cuya confianza  $\geq$  min\_confidence e interés  $\geq$  min\_interest.

Evidencia — código:

In [19]:

```
def generate_rules(freq, support, n_baskets,
                  min_confidence=0.5,
                  min_interest=0.5):
    """Genera reglas de asociacion A -> B a partir
    de itemsets frecuentes.

    Parametros
    -----
    freq : dict {k: {frozenset: int}}
        Itemsets frecuentes agrupados por tamano k.
    support : dict {frozenset: int}
        Conteo absoluto para todo itemset frecuente
        (incluyendo singletons).
    n_baskets : int
        Numero total de canastas.
    min_confidence : float, default 0.5
        Umbral minimo de confianza.
    min_interest : float, default 0.5
        Umbral minimo de interes.

    Solo se retornan reglas cuya confianza >= min_confidence
    e interes >= min_interest.

    Retorna: pd.DataFrame con columnas:
        antecedent, consequent, support, confidence,
        lift, interest.
    """
    rows = []
    for k, itemsets in freq.items():
        if k < 2:
            continue
        for I, count_I in itemsets.items():
            items = list(I)
            for r in range(1, len(items)):
                for a_tuple in combinations(items, r):
                    A = frozenset(a_tuple)
                    B = I - A
                    if not B:
                        continue
                    count_A = support[A]
                    count_B = support[B]
                    confidence = count_I / count_A
                    supp = count_I / n_baskets
                    p_b = count_B / n_baskets
                    lift = confidence / p_b
                    interest = abs(confidence - p_b)
                    if confidence >= min_confidence and interest >= min_interest:
                        rows.append({
                            "antecedent": A,
                            "consequent": B,
                            "support": supp,
                            "confidence": confidence,
                            "lift": lift,
                            "interest": interest,
                        })
    return pd.DataFrame(rows)
```

## Ejecución

Evidencia — código y salida: generación de reglas (min\_confidence=0.5, min\_interest=0.5):

```
In [20]:

# support: conteo absoluto de todo itemset frecuente (incluyendo singletons)
support = {}
for k, itemsets in freq.items():
    support.update(itemsets)

rules = generate_rules(freq, support, n_baskets, min_confidence=0.5, min_interest=0.5)
print("Número de reglas generadas:", len(rules))
assert len(rules) == 8
print("Verificación OK: 8 reglas generadas")
```

**Respuesta:** con los parámetros indicados se obtienen **8** reglas, coincidiendo con la verificación del enunciado.

Evidencia — código: reglas ordenadas por lift, con descripción de productos:

```
In [21]:

def describe_itemset(iset):
    return " + ".join(sorted(describe(i) for i in iset))

rules_sorted = rules.sort_values("lift", ascending=False).reset_index(drop=True)
rules_sorted["antecedent_desc"] = rules_sorted["antecedent"].apply(describe_itemset)
rules_sorted["consequent_desc"] = rules_sorted["consequent"].apply(describe_itemset)

display_cols = ["antecedent_desc", "consequent_desc", "support", "confidence", "lift", "interest"]
rules_sorted[display_cols]
```

| Antecedente                             | Consecuente                             | Soporte | Confianza | Lift  | Interés |
|-----------------------------------------|-----------------------------------------|---------|-----------|-------|---------|
| ROSES REGENCY<br>TEACUP AND<br>SAUCER   | GREEN REGENCY<br>TEACUP AND<br>SAUCER   | 0.0206  | 0.7047    | 27.30 | 0.6789  |
| GREEN REGENCY<br>TEACUP AND<br>SAUCER   | ROSES REGENCY<br>TEACUP AND<br>SAUCER   | 0.0206  | 0.7970    | 27.30 | 0.7678  |
| SWEETHEART<br>CERAMIC TRINKET<br>BOX    | STRAWBERRY<br>CERAMIC TRINKET<br>BOX    | 0.0234  | 0.7319    | 14.17 | 0.6802  |
| WOODEN FRAME<br>ANTIQUE WHITE           | WOODEN PICTURE<br>FRAME WHITE<br>FINISH | 0.0271  | 0.5601    | 12.38 | 0.5148  |
| WOODEN PICTURE<br>FRAME WHITE<br>FINISH | WOODEN FRAME<br>ANTIQUE WHITE           | 0.0271  | 0.5989    | 12.38 | 0.5505  |
| JUMBO BAG<br>STRAWBERRY                 | JUMBO BAG RED<br>RETROSPOT              | 0.0260  | 0.6288    | 7.07  | 0.5398  |
| JUMBO BAG PINK<br>POLKADOT              | JUMBO BAG RED<br>RETROSPOT              | 0.0290  | 0.6022    | 6.77  | 0.5132  |



| Antecedente                            | Consecuente                              | Soporte | Confianza | Lift | Interés |
|----------------------------------------|------------------------------------------|---------|-----------|------|---------|
| RED HANGING<br>HEART T-LIGHT<br>HOLDER | WHITE HANGING<br>HEART T-LIGHT<br>HOLDER | 0.0315  | 0.7082    | 5.30 | 0.5746  |

## Preguntas

¿Qué indica un valor de lift mayor a 1? Que la presencia de A hace que B aparezca **más frecuentemente** de lo que aparecería por azar (independencia estadística) — existe una asociación positiva entre A y B; cuanto mayor el lift, más fuerte la asociación (p. ej.  $\text{lift} \approx 27$  significa que comprar A hace  $\sim 27$  veces más probable comprar B que si fueran independientes).

¿Qué indica un valor de lift menor a 1? Que la presencia de A hace que B aparezca **menos frecuentemente** de lo esperado por azar — existe una asociación negativa (los productos tienden a *no* comprarse juntos, posiblemente porque son sustitutos entre sí).

Evidencia — código:

```
In [22]:  
  
strongest = rules_sorted.iloc[0]  
print("Antecedente:", strongest["antecedent_desc"])  
print("Consecuente:", strongest["consequent_desc"])  
print("Lift:", round(strongest["lift"], 2))  
print("Confianza:", round(strongest["confidence"], 4))  
print("Soporte:", round(strongest["support"], 4))
```

¿Qué par de productos tiene la asociación más fuerte? La asociación más fuerte (mayor lift) es entre **ROSES REGENCY TEACUP AND SAUCER** y **GREEN REGENCY TEACUP AND SAUCER** ( $\text{lift} \approx 27.3$ , confianza  $\approx 70.47\%$ ), es decir, los clientes que compran uno de estos productos de la colección Regency tienen una probabilidad muchísimo mayor de comprar también el otro, comparado con lo esperado si las compras fueran independientes.